

DANIYA ALI

CT –25159

COURSE CODE : CT-159 (DSA)

INSTRUCTOR : MR MUDASSIR

ADMED

LAB NO 2 (LINKED LIST)

QUESTION NO 1

SOURCE CODE:

```
Node* mergeTwoLists(Node* list1, Node* list2) {
    Node dummy(0);
    Node* current = &dummy;

    while (list1 != nullptr && list2 != nullptr) {
        if (list1->data <= list2->data) {
            current->next = list1;
            list1 = list1->next;
        } else {
            current->next = list2;
            list2 = list2->next;
        }
        current = current->next;
    }

    current->next = (list1 != nullptr) ? list1 : list2;

    return dummy.next;
}

int main(){
    vector<int>l1={1,3,5};
    vector<int>l2={2,4,6};
    Node* h1=convertArr2LL(l1);
    Node* h2=convertArr2LL(l2);
    Node* mergedHead=mergeTwoLists(h1,h2);
    print(mergedHead);
    return 0;
}
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\Users\User\OneDrive\Desktop\DSA\" ; if ($?) { g++ mergeLL.cpp -o mergeLL } ; if ($?) { .\mergeLL }
1
2
3
4
5
6
```

QUESTION NO 2

SOURCE CODE:

```
Node* deleteDuplicates(Node* head) {
    Node* current = head;

    while (current != nullptr && current->next != nullptr) {
        if (current->data == current->next->data) {
            Node* temp = current->next;
            current->next = current->next->next;
            delete temp; // Free memory for the duplicate node
        } else {
            current = current->next;
        }
    }

    return head;
}

int main(){
    vector<int>ls={1,1,2,3,4,4,5,6};
    Node* head=convertArr2LL(ls);
    head=deleteDuplicates(head);
    print(head);
    return 0;
}
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\Users\User\OneDrive\Desktop\DSA\" ; if ($?) { g++ duplicateLL.cpp
1
2
3
4
5
6
7
```

QUESTION NO 3

SOURCE CODE:

```

Node *reverseLL(Node *head)
{
    if (head == nullptr || head->next == nullptr)
        return head;

    Node *prev = NULL;

    // Start from the head of the list
    Node *curr = head;

    // Traverse the list
    while (curr != NULL)
    {
        // Save the next node
        Node *front = curr->next;

        // Reverse the current node's pointer
        curr->next = prev;

        // Move prev to current node
        prev = curr;

        // Move to the next node
        curr = front;
    }

    // Return new head (last node becomes first)
    return prev;
}

Node *middleNode(Node *head)
{
    // If the list is empty or has only
    // one element, return the head as it's the middle.
    if (head == nullptr || head->next == nullptr)
        return head;

    Node *slow = head; //move pointer one step ahead
    Node *fast = head->next; //move pointer two steps ahead

    while (fast != nullptr && fast->next != nullptr)
    {
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}

```

```

bool isPalindrome(Node *head)
{
    if (head->next == nullptr)
        return true;

    Node *mid = middleNode(head);

    // Reverse the second half of the linked list starting from the middle
    Node *newHead = reverseLL(mid->next);

    // Pointer to the reversed second half
    Node *second = newHead;

    // Pointer to the first half
    Node *first = head;

    // Compare data values of nodes from both halves
    while (second != nullptr)
    {
        // If values do not match, the list is not a palindrome
        if (first->data != second->data)
        {
            reverseLL(newHead); // Reverse the second half back to its original state
            return false;
        }
        first = first->next;
        second = second->next;
    }

    // Reverse the second half back to its original state
    reverseLL(newHead);
    return true;
}

int main()
{
    // Create a linked list with values 1, 5, 2, 5, and 1 (15251, a palindrome)
    Node *head = new Node(1);
    head->next = new Node(5);
    head->next->next = new Node(2);
    head->next->next->next = new Node(5);
    head->next->next->next->next = new Node(1);
}

```

```

// Print the original linked list
cout << "Original Linked List: ";
print(head);

// Check if the linked list is a palindrome
if (isPalindrome(head))
{
    cout << "The linked list is a palindrome." << endl;
}
else
{
    cout << "The linked list is not a palindrome." << endl;
}

return 0;

```

OUTPUT:

```

-----
PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\Users\User\OneDrive\Desktop\DSA"
Original Linked List: 1
5
2
5
1
The linked list is a palindrome.

```

QUESTION NO 4

SOURCE CODE:

```
#include<bits/stdc++.h>
using namespace std;
class CircularQueue {
private:
    struct Node {
        int data;
        Node* next;
        Node(int val) : data(val), next(nullptr) {}
    };

    Node* front;
    Node* rear;

public:
    CircularQueue() {
        front = nullptr;
        rear = nullptr;
    }

    bool isEmpty() {
        return front == nullptr;
    }

    void enqueue(int val) {
        Node* newNode = new Node(val);
        if (isEmpty()) {
            front = rear = newNode;
            rear->next = front;
        } else {
            rear->next = newNode;
            rear = newNode;
            rear->next = front; // Complete the circle back to front
        }
    }

    int dequeue() {
        if (isEmpty()) {
```

```

        return -1;
    }

    int poppedVal = front->data;

    if (front == rear) {
        delete front;
        front = rear = nullptr;
    } else {
        Node* temp = front;
        front = front->next;
        rear->next = front; // Update the circular link
        delete temp;
    }

    return poppedVal;
}

int peek() {
    if (isEmpty()) {
        return -1;
    }
    return front->data;
}
};

```

```

int main() {
    CircularQueue q;

    cout << "Is queue empty? " << (q.isEmpty() ? "Yes" : "No") << endl;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    cout << "Front element (peek): " << q.peek() << endl;

    cout << "Dequeued: " << q.dequeue() << endl;
    cout << "Dequeued: " << q.dequeue() << endl;

    q.enqueue(40);
    q.enqueue(50);

    cout << "Front element after adding more: " << q.peek() << endl;

    cout << "Clearing the queue:" << endl;
    while (!q.isEmpty()) {
        cout << "Dequeued: " << q.dequeue() << endl;
    }

    cout << "Is queue empty now? " << (q.isEmpty() ? "Yes" : "No") << endl;

    return 0;
}

```

OUTPUT:

```

PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\Users\User\OneDrive\Desktop\DSA\"
Is queue empty? Yes
Front element (peek): 10
Dequeued: 10
Dequeued: 20
Front element after adding more: 30
Clearing the queue:
Dequeued: 30
Dequeued: 40
Dequeued: 50
Is queue empty now? Yes

```

QUESTION NO 5

SOURCE CODE:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // Node structure
5  struct Node {
6      int val;
7      Node *next;
8      Node(int d=0) {
9          val = d;
10         next = NULL;
11     }
12 };
13
14 // Class to represent stack
15 class LinkedListStack {
16 private:
17     Node *head;
18     int size;
19
20 public:
21
22     LinkedListStack() {
23         head = NULL;
24         size = 0;
25     }
26
27     // Method to push an element onto the stack
28     void push(int x) {
29
30         Node *element = new Node(x);
31
32         element->next = head;
33         head = element;
34
35         size++;
36     }
37
38     // Method to pop an element from the stack
39     int pop() {
40
41         if (head == NULL) {
42             return -1;
43         }
44
45         int value = head->val;
```

```

46     Node *temp = head;
47     head = head->next;
48     delete temp;
49     size--;
50
51     return value;
52 }
53
54 // Method to get the top element of the stack
55 int top() {
56
57     if (head == NULL) {
58         return -1;
59     }
60
61     return head->val;
62 }
63
64 // Method to check if the stack is empty
65 bool isEmpty() {
66     return (size == 0);
67 }
68 };
69
70 int main() {
71
72     LinkedListStack st;
73
74     vector<string> commands = {"LinkedListStack", "push", "push",
75                               "pop", "top", "isEmpty"};
76     // List of inputs
77     vector<vector<int>> inputs = {{}, {3}, {7}, {}, {}, {}};
78
79     for (int i = 0; i < commands.size(); ++i) {
80         if (commands[i] == "push") {
81             st.push(inputs[i][0]);
82             cout << "null ";
83         } else if (commands[i] == "pop") {
84             cout << st.pop() << " ";
85         } else if (commands[i] == "top") {
86             cout << st.top() << " ";
87         } else if (commands[i] == "isEmpty") {
88             cout << (st.isEmpty() ? "true" : "false") << " ";

```

OUTPUT:

```

PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\Users\User\OneDrive\Desktop\DSA"
null null null 7 3 false

```